SIMATS ENGINEERING

CSA0705 - COMPUTER NETWORKS

STUDENT NAME : BALAJI A

REGISTER NO : 1911260245

DEPARTMENT : B.TECH COMPUTER SCIENCE ENGINEERING

ACADEMIC YEAR : 2026

FACULTY NAME : KUMARAN G

NATURE OF ASSIGNMENT : INDIVIDUAL

1. Introduction

A CRC with generator 1011 ($x^3 + x + 1$) protects frames between two shop-floor switches. This report answers all six parts of Task 7: the CRC calculation, verification of a received frame, burst-error detection, two-dimensional parity, the Internet checksum, and a comparison of simple even parity with CRC-3. All answers are worked out step by step, and each result was checked again by recomputing it.

2. CRC Calculation

Data: `1101011011` **Generator:** `1011` (degree 3)

2.1 Why three zeros are appended

The generator has degree 3, so the CRC remainder has 3 bits. Appending 3 zeros to the data (multiplying it by x^3) makes room for these 3 CRC bits. After the division, the remainder replaces the zeros, so the final codeword is exactly divisible by the generator.

2.2 Modulo-2 division

Dividend = data + 3 zeros = `1101011011000` (13 bits). At each step, if the leading bit is 1, XOR with `1011`; if it is 0, XOR with `0000`. The leftmost bit is dropped, and the next bit is brought down.

Step	Current 4 bits	XOR with	Result	Remainder (3 bits)	Bring down
1	1101	1011	0110	110	0
2	1100	1011	0111	111	1
3	1111	1011	0100	100	1
4	1001	1011	0010	010	0
5	0100	0000	0100	100	1
6	1001	1011	0010	010	1
7	0101	0000	0101	101	0
8	1010	1011	0001	001	0
9	0010	0000	0010	010	0
10	0100	0000	0100	100	–

2.3 Result

CRC remainder = `100` Transmitted codeword = `1101011011100` = `1101011011100`

Check: dividing `1101011011100` by `1011` gives remainder `000`, so the codeword is valid.

3. Received Frame Verification

Received frame: `1101011011101` (13 bits). It is divided by `1011` using the same method.

Step	Current 4 bits	XOR with	Result	Remainder (3 bits)	Bring down
1	1101	1011	0110	110	0
2	1100	1011	0111	111	1
3	1111	1011	0100	100	1
4	1001	1011	0010	010	0
5	0100	0000	0100	100	1
6	1001	1011	0010	010	1
7	0101	0000	0101	101	1
8	1011	1011	0000	000	0
9	0000	0000	0000	000	1
10	0001	0000	0001	001	–

Final remainder = 001 (not zero) → the receiver REJECTS the frame.

Explanation: A correct frame leaves remainder 000. The remainder 001 shows the frame was changed in transit; compared with the transmitted codeword 1101011011100, the last bit has flipped (0 → 1), and the CRC detects it.

4. Burst-Error Detection

4.1 What a burst error is

A burst error is a group of errors that occur close together. Its **length** is counted from the first flipped bit to the last flipped bit, including the bits in between (those in between may or may not be flipped). For example, the error pattern 1011 has length 4, but only 3 bits are flipped.

4.2 Maximum burst length guaranteed to be detected

A CRC generator of degree r detects **every burst error of length ≤ r**. For 1011 (degree 3), all bursts of length **up to 3 bits** are guaranteed to be detected.

Reason: a burst of length 3 or less, once shifted to the end of the frame, is a non-zero polynomial of degree at most 2 (times a power of x). Such a polynomial is smaller than the degree-3 generator and cannot be a multiple of it, so the remainder is never zero.

4.3 A valid undetectable error pattern

An error is undetected only if its polynomial is a multiple of the generator. The simplest one is the generator itself:

$$E(x) = x^3 + x + 1 \quad \rightarrow \quad 1011$$

Placed at the end of the 13-bit codeword, the error pattern is 0000000001011 (length 4, 3 bits flipped).

Divisibility check: 1011 ÷ 1011 leaves remainder 000.

```
Sent codeword     1101011011100
Error pattern  ⊕  0000000001011
Received frame    1101011010111
```

Dividing the received frame `1101011010111` by `1011`:

Step	Current 4 bits	XOR with	Result	Remainder (3 bits)	Bring down
1	1101	1011	0110	110	0
2	1100	1011	0111	111	1
3	1111	1011	0100	100	1
4	1001	1011	0010	010	0
5	0100	0000	0100	100	1
6	1001	1011	0010	010	0
7	0100	0000	0100	100	1
8	1001	1011	0010	010	1
9	0101	0000	0101	101	1
10	1011	1011	0000	000	–

Remainder = `000` → the corrupted frame is wrongly ACCEPTED (error not detected).

4.4 Why it is not detected

The received frame is R(x) = T(x) + E(x). T(x) is divisible by the generator because it is a valid codeword, and E(x) = 1011 is also divisible by the generator. The sum of two multiples of the generator is again a multiple, so the remainder is 000 and the receiver sees nothing wrong. Note that this pattern has length 4, which is more than the guaranteed length of 3.

5. Two-Dimensional Parity

5.1 Original 4 × 4 data block

	C1	C2	C3	C4
Row 1	1	0	1	1
Row 2	0	1	1	0
Row 3	1	1	0	0
Row 4	0	0	1	1

5.2 Even parity for rows and columns

Even parity bit = 0 if the number of 1s is even, and 1 if it is odd (so that the total number of 1s becomes even).

Row	Number of 1s	Row parity bit
Row 1	1 0 1 1 → 3 ones	**1**
Row 2	0 1 1 0 → 2 ones	**0**
Row 3	1 1 0 0 → 2 ones	**0**
Row 4	0 0 1 1 → 2 ones	**0**

Column	Number of 1s	Column parity bit
C1	1 0 1 0 → 2 ones	**0**
C2	0 1 1 0 → 2 ones	**0**
C3	1 1 0 1 → 3 ones	**1**
C4	1 0 0 1 → 2 ones	**0**

5.3 Final table with parity row and column

	C1	C2	C3	C4	Row parity
Row 1	1	0	1	1	**1**
Row 2	0	1	1	0	**0**
Row 3	1	1	0	0	**0**
Row 4	0	0	1	1	**0**
Column parity	**0**	**0**	**1**	**0**	**1**

5.4 Verification

- Each row including its parity bit has an even number of 1s: `1011|1 (4)`, `0110|0 (2)`, `1100|0 (2)`, `0011|0 (2)`.
- Each column including its parity bit has an even number of 1s: C1 = 2, C2 = 2, C3 = 4 (1+1+0+1+1), C4 = 2.
- Corner bit = parity of the row-parity bits (1+0+0+0 = 1) = parity of the column-parity bits (0+0+1+0 = 1) = **1**. Both agree, so the table is consistent.

5.5 A 4-bit rectangular error that is not detected

Flip the four bits at the corners of a rectangle: Row 1 and Row 2, Columns 1 and 2 (positions `(1,1)` `(1,2)` `(2,1)` `(2,2)`).

After error	C1	C2	C3	C4	Row parity
Row 1	**0**	**1**	1	1	**1**
Row 2	**1**	**0**	1	0	**0**
Row 3	1	1	0	0	**0**
Row 4	0	0	1	1	**0**
Column parity	**0**	**0**	**1**	**0**	**1**

Why it is missed: Row 1 and Row 2 each have two flipped bits, so their number of 1s changes by 0 or ±2 and stays even. Columns 1 and 2 also have two flipped bits each, so their parity is unchanged too. Every received row and column parity still matches the received parity bits, so the receiver finds no error. (The highlighted cells show the flipped bits.)

6. Internet Checksum

Words: `4500  0073  0000  4000`

6.1 One's-complement addition and end-around carry

The Internet checksum adds the 16-bit words as ordinary binary numbers. If the sum is larger than 16 bits (a carry out of the leftmost bit), that carry is not thrown away; it is added back to the lowest bit of the sum. This is called end-around carry, and it is how 16-bit one's-complement addition works. The checksum is then the one's complement (every bit flipped) of the final 16-bit sum.

6.2 Step-by-step hexadecimal addition

Step	Addition	Result	Carry beyond 16 bits
1	`4500 + 0073`	`4573`	None
2	`4573 + 0000`	`4573`	None
3	`4573 + 4000`	`8573`	None

No carry came out of the 16th bit in any step, so no end-around carry is needed here. **Final sum = `8573`.**

6.3 One's complement of the sum

```
8573 (hex) = 1000 0101 0111 0011 (binary)
Flip bits  = 0111 1010 1000 1100 (binary) = 7A8C (hex)
```

Hex check: `FFFF − 8573 = 7A8C`.

Final sum = `8573` Internet checksum (one's complement) = `7A8C`

Receiver check: `8573 + 7A8C = FFFF`, whose one's complement is 0000, so an unchanged message is accepted.

7. Simple Even Parity vs CRC-3

7.1 Overview

	Simple even parity (1 bit)	CRC-3 (generator 1011)
Detects	Any odd number of flipped bits (1, 3, 5 …)	All single-bit errors, all burst errors of length ≤ 3, and 2-bit errors that are not exactly 7 (or a multiple of 7) positions apart
Misses	Any even number of flipped bits (2, 4, 6 …), wherever they are	Error patterns that are multiples of 1011 (e.g. 1011, 11101, and 2-bit errors 7 positions apart)

Length vs flipped bits: the **length** of an error pattern is the distance from its first to its last flipped bit (counting both ends); the **number of flipped bits** is just how many 1s it has. For example, `10001` has length 5 but only 2 flipped bits.

7.2 Test frames

- **Parity:** data `1101011011` contains seven 1s, so the even-parity bit is 1, giving the frame `11010110111` (eight 1s).
- **CRC-3:** codeword from Section 2, `1101011011100`. A frame is accepted if the remainder after dividing by 1011 is 000.

7.3 Error examples

#	Scheme	Error pattern	Length	Flipped bits	Result	Reason
1	Even parity	`00000000011`	2	2	**MISSED**	Two flips: 8 ones → 6 ones, still even.
2	Even parity	`00000001111`	4	4	**MISSED**	Four flips: 8 ones → 6 ones, still even.
3	Even parity	`00000001011`	4	3	**DETECTED**	Three flips (odd): 8 ones → 7 ones, parity broken.
4	CRC-3	`0000000000011`	2	2	**DETECTED**	E = 11 is not a multiple of 1011; remainder 011.
5	CRC-3	`0000010000001`	8	2	**MISSED**	$x^7 + 1 = (x+1)(x^3+x+1)(x^3+x^2+1)$ is divisible by 1011; remainder 000.
6	CRC-3	`0000000001111`	4	4	**DETECTED**	1111 is not a multiple of 1011; remainder 100.
7	CRC-3	`0000000011101`	5	4	**MISSED**	11101 = 1011 × 11 (modulo 2), a multiple; remainder 000.
8	CRC-3	`0000000001011`	4	3	**MISSED**	1011 is the generator itself; remainder 000.

Examples 1–3 are applied to the 11-bit parity frame; examples 4–8 are applied to the 13-bit CRC codeword. Each CRC result was obtained by dividing the error pattern (and the corrupted frame) by `1011`; a remainder of `000` means the error is missed.

7.4 An important difference

Examples 3 and 8 use almost the same error: three flipped bits, error pattern `1011`. Simple even parity **detects** it, because three is an odd number of flips and the parity changes. CRC-3 **misses** it, because 1011 is a multiple of the generator, so the remainder is 000. So parity can catch some odd-bit errors that CRC-3 misses. The reverse is also true: parity misses every 2-bit error (example 1), while CRC-3 catches adjacent 2-bit errors (example 4).

8. Conclusion

- CRC for data 1101011011 with generator 1011 is **100**; the transmitted codeword is **1101011011100**.
- The received frame 1101011011101 leaves remainder **001**, so it is **rejected**.
- A degree-3 CRC detects all bursts of length up to **3**; the pattern 1011 is a multiple of the generator and goes undetected.
- Two-dimensional parity detects any single-bit error but misses a rectangle of four flipped bits.
- The Internet checksum of the four words is **7A8C** (sum 8573).
- CRC-3 is stronger against burst errors than simple parity, but neither method detects every error.